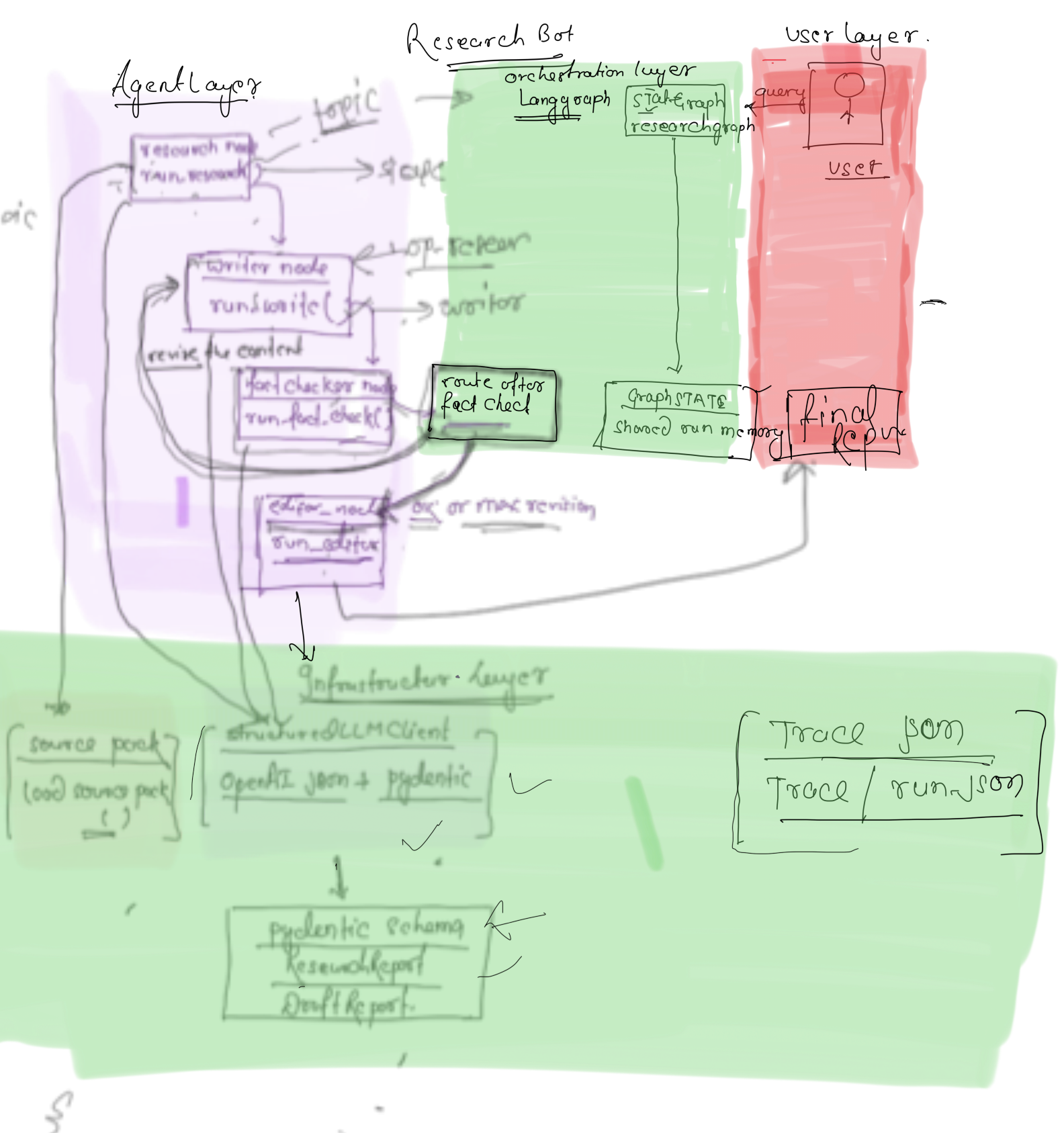
Langraph project Excution

research_op:
Research_json
{
research_topic
tin

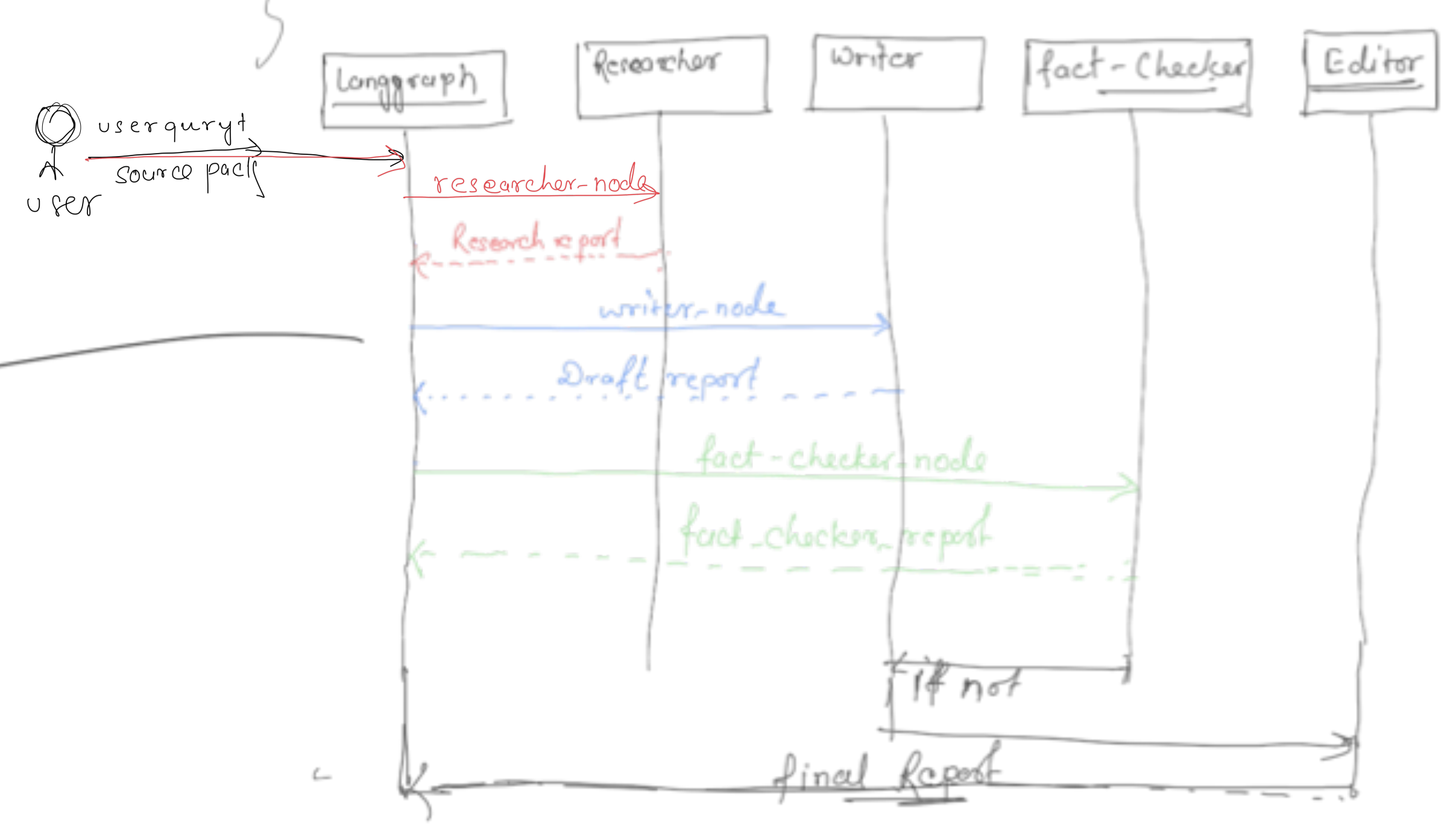
}
{

}


Research Bot
User layer.
Agent Layer
topic
orchestration layer
Langgraph
StateGraph
researchgraph
query
user
research node
run_research
state
writer node
runwrite()
op_research
writer
revise the content
fact checker node
run_fact_check()
route after fact check
GraphSTATE
shared run memory
final Report
editor_node
run_editor
ok or max revition
Infrastructure layer
source pack
load source pack()
StructuredLLMClient
OpenAI json + pydentic
Trace json
Trace / run_json
Pydentic schema
ResearchReport
DraftReport.

{
titel
writer
timestamp
content
Refe

}


Langgraph
Researcher
Writer
fact-Checker
Editor
userqury+
source pack
user
researcher-node
Research report
writer-node
Draft report
fact-checker-node
fact_checker report
If not
final Report

Research Report output

1. topic : Auth 2
2. findings: {
   claim
   evidences: [
   {
   source_id :
   titel :
   snippet :
   releven scor
   }
   {

   }
   ]
3. unresolved_question
4. overall_confidence :

⑥ Agent functions

Each Agent will have run, it will be responsi to run the perticuler Agent.

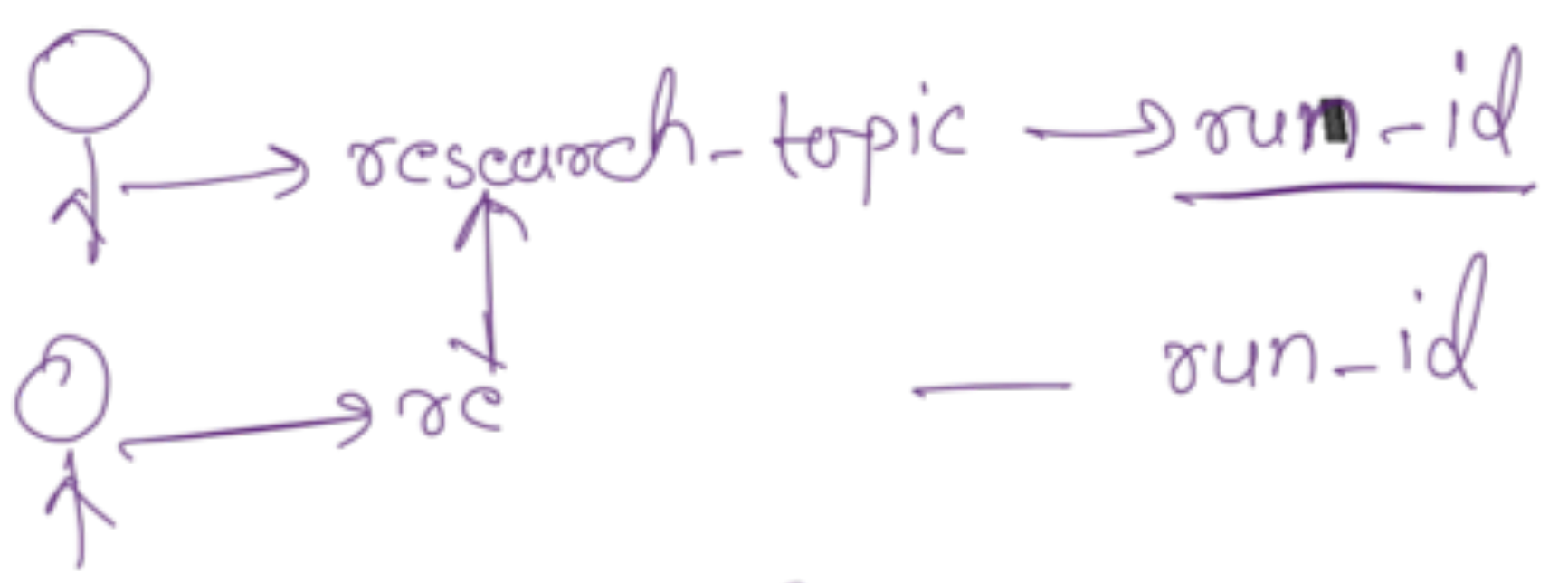
		Role	input	output
①.	run_research →	find evidence	query + source	Research Report
②-	run-writer →	Draft report	query + research(①) optional critique ③	Draft Report.
③	runfact-checker →	verify the claims	draft ② + research ①	Factcheck Report.
④	run editor →	polish final Report	draft ② + fact checker rep ③	FinalReport

function References - quick lookup

⑥ function	layer	what it does
① load_source_pack()	Infrastructure lay r	Reads all the md file
② render_source_for_prm()	"	format sources into Researcher prompt text
③ schema_instruction()	"	Apend the json schema
④ structured LLm clien. completion()	"	openAcall + pydantic validation #repair
⑤ run_research	Agent	
⑥ run_writter	"	
⑦ run_fact_checker	"	
⑧ run_editor	"	

```
def make_message(
    sender: AgentRole,
    receiver: AgentRole,
    message_type: MessageType,
    task: str,
    payload_model: BaseModel,
    confidence: float,
) -> AgentMessage:
```

What Should be there in state

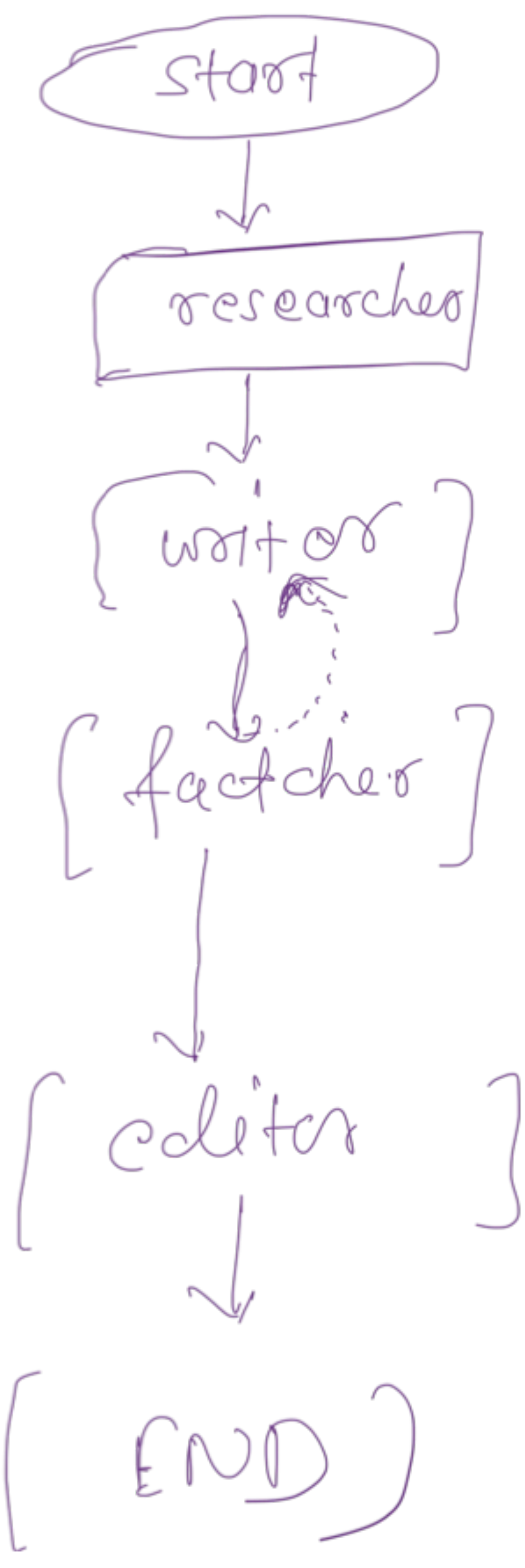

research_topic
run_id
rc
run_id

State → {

run_id : str,
user_query : str,
documents : list[sourceDocuments]
source_id : list[str]
research_report : ResearchReport | None
draft_report : DraftReport | None
fact_checker_repo : FackcheckerReport | None
final_report : FinalReport | None
✓ revision_count : int
max_revision_count : int
Status : str # created | running | need_revision | Completed
messages : Annotated[list[AgentMessage], operator.Add]
error : Annotated[list[str]], operator.add]

}

Flow the graph flow will look like


start
researcher
writer
factcher
editor
END